



TP 1

Environnement, Git et les limites du traitement local

Big Data Engineering — Master 1 — TP (2 h 30)

Prof. Samba Ndiaye • DMI/FST – UCAD
samba.ndiaye@ucad.edu.sn

Objectifs et livrable du TP

À la fin de ce TP, vous aurez :

- > un **environnement de travail opérationnel** : Python, Git, GitHub, Colab ;
- > le **dépôt Git individuel** du cours, initialisé proprement — il portera *tous* vos livrables du semestre ;
- > généré le **jeu de données fil rouge** sur votre machine ;
- > mesuré **expérimentalement** les limites de Pandas en local ;
- > rédigé un premier **diagnostic technique** argumenté.

[!] Livrable (à pousser sur GitHub avant la séance 2)

Dépôt initialisé (structure imposée, README.md complété) + docs/DIAGNOSTIC.md : pourquoi Pandas ne suffit plus sur ces données, et ce qu'une architecture distribuée doit apporter. Notebook d'exploration complété dans notebooks/.

Déroulé de la séance (2 h 30)

Partie	Contenu	Durée
A	Installation et vérification de l'environnement	30 min
B	Création du dépôt Git individuel	20 min
C	Génération du jeu de données fil rouge	15 min
D	Exploration avec Pandas (notebook guidé)	45 min
E	La montée en charge... jusqu'au mur	25 min
F	Discussion guidée et quiz	15 min

Matériel fourni sur le dépôt du cours :

- > `generate_data.py`, `requirements.txt` (générateur de données);
- > `check_env.py` (vérification automatique de l'environnement);
- > `TP1_exploration.ipynb` (notebook guidé à compléter);
- > `depot_etudiant_squelette.zip` (structure du dépôt).

A.1 — Installer et vérifier Python

Prérequis : **Python 3.9 ou plus récent**, pip, et venv.

```
# Verifier les versions installees
```

```
python3 --version          # >= 3.9
```

```
pip3 --version
```

```
# Creer et activer un environnement virtuel pour le cours
```

```
python3 -m venv venv-bigdata
```

```
source venv-bigdata/bin/activate      # Linux / macOS
```

```
venv-bigdata\Scripts\activate        # Windows (PowerShell)
```

```
# Installer les dependances du TP
```

```
pip install -r requirements.txt
```

```
pip install pandas jupyter
```

/!\ Un environnement virtuel par projet

Travailler dans le Python « système » finit toujours par des conflits de versions. Le réflexe venv est exigé toute l'année.

A.2 — Git, GitHub et Colab

1. Installer Git et vous identifier :

```
git --version  
git config --global user.name "Prenom NOM"  
git config --global user.email "votre.email@..."
```

2. Créer un compte **GitHub** (si besoin) avec votre email étudiant, et configurer l'authentification (token ou clé SSH).
3. Vérifier votre accès **Google Colab** : ouvrir un notebook, exécuter une cellule. Colab servira pour les démonstrations à l'échelle 1.0 (données trop volumineuses pour un laptop).

[!] Validation automatique

Exécutez `python3 check_env.py` : le script contrôle Python, pip, Git, et les bibliothèques requises, puis affiche un bilan. **Tout doit être vert avant de passer à la partie B.** Appelez l'enseignant en cas de blocage.

B.1 — Créer le dépôt individuel

1. Sur GitHub : créer un dépôt **privé** nommé bigdata-ucad-2026-<prenom-nom>.
2. Inviter l'enseignant en collaborateur (identifiant donné en séance).
3. En local : décompresser depot_etudiant_squelette.zip, puis :

```
cd bigdata-ucad-2026-<prenom-nom>
git init
git add .
git commit -m "Initialisation du depot du cours"
git branch -M main
git remote add origin <URL de votre depot>
git push -u origin main
```

/!\ Le dépôt est votre copie d'examen permanente

Tous les livrables et jalons du semestre passent par ce dépôt. Un travail non poussé sur GitHub est un travail **inexistant**.

B.2 — La structure imposée

```
bigdata-ucad-2026-<prenom-nom>/  
|-- README.md           # a completer : qui, quoi, comment executer  
|-- .gitignore          # fourni : exclut les donnees generees  
|-- requirements.txt  
|-- check_env.py  
|-- data/               # donnees generees -- JAMAIS commitees  
|-- notebooks/          # vos notebooks de TP  
|-- src/                # plus tard : code Python modulaire  
|-- docs/  
    |-- DIAGNOSTIC.md   # livrable de la seance 1
```

/!\ Pourquoi data/ est dans le .gitignore

events.json pèse $\approx 0,7$ Go à l'échelle 1.0 : GitHub refuse les fichiers de plus de 100 Mo, et versionner des données **régénérables** n'a aucun sens — le script + la graine suffisent à les reproduire à l'identique. On versionne le *code*, pas les données.

C.1 — Génération des données

Fil rouge Le fil rouge : une plateforme e-commerce sénégalaise fictive (24 mois d'activité, villes du Sénégal, paiements mobile money, montants en FCFA).

```
# Echelle 0.1 -- adaptee a un laptop 8 Go (~90 Mo, < 1 min)
python3 generate_data.py --scale 0.1 --outdir ./data
```

- > La génération est **déterministe** (graine fixe = 42) : toute la promotion obtient *exactement* les mêmes fichiers — vos résultats sont donc directement comparables (et vérifiables).
- > L'échelle **1.0** (50 000 clients, 500 000 commandes, $\approx$ 860 Mo) sera utilisée sur **Colab** en partie E — ne l'exécutez pas sur votre laptop pendant le TP.

[!] Vérification

```
ls -lh data/ doit lister 6 fichiers : 4 CSV (customers, products, orders,
order_items) et 2 JSON Lines (payments, events).
```

C.2 — Les six fichiers du fil rouge

Fichier	Format	Contenu	Lignes (0.1)
customers.csv	CSV	clients (villes, téléphones...)	5 025
products.csv	CSV	catalogue, prix en FCFA	≈ 630
orders.csv	CSV	commandes sur 24 mois	50 000
order_items.csv	CSV	lignes de commande	≈ 113 000
payments.json	JSON Lines	paiements (OM, Wave...)	≈ 44 600
events.json	JSON Lines	clics, paniers, achats	≈ 330 000

- > À l'échelle 1.0, multipliez par ≈ 10 : `events.json` atteint **3,3 millions de lignes** (≈ 685 Mo).
- > Ces données sont **volontairement imparfaites** : n'en tirez pas de conclusions définitives aujourd'hui — le nettoyage commence en séance 3.

D — Le notebook guidé

Ouvrez `notebooks/TP1_exploration.ipynb` (Jupyter local). Le notebook vous guide; complétez les cellules marquées **À COMPLÉTER**.

1. **Exercice 1** — charger `customers.csv` : dimensions, types des colonnes, aperçu, empreinte mémoire (`memory_usage(deep=True)`).
2. **Exercice 2** — charger `orders.csv` en mesurant le **temps de chargement**; premières statistiques (commandes par statut, par canal).
3. **Exercice 3** — jointure Pandas `orders` $\times$ `order_items` : mesurer mémoire *avant* / *après*.
4. **Exercice 4** — charger `events.json` (`lines=True`) : temps, mémoire, ratio taille mémoire / taille disque.
5. **Exercice 5** — consigner toutes vos mesures dans le **tableau de relevés** (il servira en partie E).

Réflexe d'ingénieur : on ne dit pas « c'est lent », on dit « **12,4 s** pour 50 000 lignes » — mesurez tout.

D — Premier contact avec la véracité

En explorant, vous remarquerez des choses étranges. **Notez-les** (sans les corriger) dans la dernière section du notebook :

- > des emails vides ou valant « N/A » dans `customers.csv` ;
- > des villes écrites de plusieurs façons (« DAKAR », « dakar », « Thies ») ;
- > des montants de `orders.csv` qui empêchent un calcul numérique direct... regardez le `dtype` de la colonne ;
- > des dates de naissance suspectes.

[!] C'est normal — et c'est voulu

Ces anomalies sont injectées délibérément dans le générateur. Elles sont l'objet des séances 3 (nettoyage) et 9 (qualité automatisée). Aujourd'hui, on **observe et on documente** ; on ne corrige pas.

E.1 — Monter en charge : échelle 1.0 sur Colab

Sur **Google Colab** (RAM affichée : ≈ 12 Go en version gratuite) :

1. téléverser `generate_data.py` et `requirements.txt`, installer les dépendances ;
2. générer l'échelle **1.0** (`-scale 1.0`, ≈ 2 min, 860 Mo) ;
3. rejouer les exercices 2 à 4 du notebook sur ces données : relever **temps** et **mémoire** dans le tableau ;
4. tenter le chargement intégral de `events.json` (≈ 685 Mo sur disque)... et observer.

/!\ Ce qui va (probablement) se passer

Le DataFrame en mémoire occupe **2 à 5 fois** la taille du fichier sur disque (objets Python, index, chaînes). Selon la session Colab : saturation RAM, avertissements, voire **crash du noyau**. Ce n'est pas un échec de votre part — c'est *la mesure* que nous cherchions.

E.2 — Extrapoler : et si c'était la production ?

Complétez le tableau d'extrapolation du notebook :

Scénario	events.json	Temps estimé	Mémoire estimée
TP échelle 0.1 (mesuré)	≈ 70 Mo	<i>vos mesures</i>	<i>vos mesures</i>
TP échelle 1.0 (mesuré)	≈ 685 Mo	<i>vos mesures</i>	<i>vos mesures</i>
Plateforme réelle, 1 an	≈ 50 Go	?	?
Grand acteur régional	≈ 1 To	?	?

Questions à traiter dans le diagnostic :

- > L'extrapolation linéaire est-elle optimiste ou pessimiste ? Pourquoi ?
- > Quelles solutions *sans* distribution connaissez-vous (échantillonnage, chunks, dtypes optimisés, formats binaires) — et jusqu'où repoussent-elles le mur ?
- > À partir de quel point le **scale-out** devient-il inévitable ?

F.1 — Discussion guidée

En groupes de 3–4, puis restitution (15 min) :

1. Votre laptop a échoué (ou peiné) sur 685 Mo. Or « Big Data » commence aux téraoctets. **Le problème est-il la taille des données, ou l'architecture ?**
2. Acheter une machine à 128 Go de RAM réglerait le TP d'aujourd'hui. Pourquoi n'est-ce **pas** une stratégie d'entreprise viable ? (coût, croissance, pannes, concurrence d'accès)
3. Dans le cycle de vie vu ce matin, **où** se situe exactement ce que nous avons fait aujourd'hui ? Qu'est-ce qui manque pour parler de « pipeline » ?
4. Pandas doit-il être jeté ? Citez des cas où il reste **le bon outil**.

Les conclusions de votre groupe alimentent directement votre DIAGNOSTIC.md.

F.2 — Quiz éclair

1. Citez les 5V et donnez, pour chacun, une illustration tirée du fil rouge généré aujourd'hui.
2. `events.json` est-il structuré, semi-structuré ou non structuré ? Justifiez.
3. Le rapport quotidien des ventes de la plateforme : batch ou streaming ? Et le blocage d'un paiement frauduleux ?
4. Pourquoi le DataFrame occupe-t-il plus de mémoire que le fichier sur disque ?
5. Pourquoi exclut-on `data/` du contrôle de version alors qu'on y passe la moitié du TP ?

[!] Auto-évaluation

Si une question résiste, relisez la partie correspondante du cours du matin *avant* de rédiger le diagnostic — il mobilise exactement ces notions.

Le livrable, en détail

À pousser sur GitHub **avant la séance 2** :

1. **Dépôt conforme** à la structure imposée, README.md complété (votre nom, description, instructions d'exécution).
2. notebooks/TP1_exploration.ipynb complété, exécuté de bout en bout (cellules de sortie visibles), tableau de relevés rempli.
3. docs/DIAGNOSTIC.md (1 à 2 pages) suivant le canevas fourni : (i) constats chiffrés — vos mesures ; (ii) analyse — pourquoi ça casse ; (iii) besoins — ce qu'une architecture distribuée doit apporter.

Critère d'évaluation (indicatif)	Poids
Dépôt propre : structure, README, messages de commit	30 %
Notebook exécuté, mesures complètes et exactes	30 %
Diagnostic : argumentation chiffrée, lucidité technique	40 %

Avant de partir — checklist

- ☐ `check_env.py` passe entièrement au vert.
- ☐ Dépôt GitHub privé créé, enseignant invité, premier push effectué.
- ☐ Données générées à l'échelle 0.1 ; `data/` bien ignoré par Git (`git status` ne les liste pas).
- ☐ Notebook entamé, tableau de relevés au moins partiellement rempli.
- ☐ Notes de la discussion guidée conservées pour le diagnostic.

[!] Pour la séance 2 — Calcul distribué : de MapReduce à Spark

Terminez le livrable, et lisez Reis & Housley, *Fundamentals of Data Engineering*, chap. 1–2. Nous répondrons à la question laissée ouverte aujourd'hui : *comment* plusieurs machines coopèrent-elles ?



Université Cheikh Anta Diop de Dakar

DMI/FST – Prof. Samba Ndiaye

samba.ndiaye@ucad.edu.sn

Références principales : J. Reis & M. Housley, Fundamentals of Data Engineering (O'Reilly, 2022)

M. Kleppmann, Designing Data-Intensive Applications (O'Reilly, 2017)

J. Damji et al., Learning Spark, 2^e éd. (O'Reilly, 2020)